

Core Language Working Group "tentatively ready" Issues for the November, 2016 (Issaquah) meeting

Section references in this document reflect the section numbering of document [WG21 N4582](#).

2094. Trivial copy/move constructor for class with volatile member

Section: 12.8 [class.copy] **Status:** tentatively ready **Submitter:** Daveed Vandevoorde **Date:** 2015-03-06

The resolution of issue 496 included the addition of 12.8 [class.copy] paragraph 25.2, making a class's copy/move constructor non-trivial if it has a non-static data member of volatile-qualified type. This change breaks the IA-64 ABI, so it has been requested that CWG reconsider this aspect of the resolution.

On a related note, the resolution of issue 496 also changed 3.9 [basic.types] paragraph 9, which makes volatile-qualified scalar types “trivial” but not “trivially copyable.” It is not clear why there is a distinction made here; the only actual use of “trivial type” in the Standard appears to be in the description of `qsort`, which should probably use “trivially copyable.” (See also issue 1746.)

Notes from the February, 2016 meeting:

CWG agreed with the suggested direction for the changes in 12.8 [class.copy]; the use of “trivial” will be dealt with separately and not as part of the resolution of this issue.

Proposed resolution (June, 2016):

1. Change 3.9 [basic.types] paragraph 9 as follows:

...called POD types. Cv-unqualified scalar types, trivially copyable class types (Clause 9 [class]), arrays of such types, and ~~non-volatile const-qualified~~ **cv-qualified** versions of these types (3.9.3 [basic.type.qualifier]) are collectively called *trivially copyable types*. Scalar types...

2. Delete bullet 12.2 of 12.8 [class.copy]:

A copy/move constructor for class `x` is trivial if it is not user-provided, its parameter-type-list is equivalent to the parameter-type-list of an implicit declaration, and if

- ...
- ~~class `x` has no non-static data members of volatile-qualified type, and~~
- ...

3. Delete bullet 25.2 of 12.8 [class.copy]:

A copy/move assignment operator for class `x` is trivial if it is not user-provided, its parameter-type-list is equivalent to the parameter-type-list of an implicit declaration, and if

- ...
- ~~class `x` has no non-static data members of volatile-qualified type, and~~

2100. Value-dependent address of static data member of class template

According to bullet 2.4 of 14.6.2.3 [temp.dep.constexpr], an *id-expression* is value-dependent if

- it names a static data member that is a dependent member of the current instantiation and is not initialized in a *member-declarator*,

This implies that the address of an initialized static data member is not value-dependent, which is incorrect.

Proposed resolution (June, 2016):

Change 14.6.2.3 [temp.dep.constexpr] paragraph 5 as follows:

An expression of the form *&qualified-id* where the *qualified-id* names a dependent member of the current instantiation is value-dependent. **An expression of the form**

&cast-expression

is also value-dependent if evaluating *cast-expression* as a core constant expression (5.20 [expr.const]) succeeds and the result of the evaluation refers to a templated entity that is an object with static or thread storage duration or a member function.